VNUHCM - UNIVERSITY OF SCIENCE
ADVANCED PROGRAM IN COMPUTER SCIENCE

ZOMBIE HOUSE 3D

Final Project Report

CS427 - 3D Visualization and Game Development

Lecturer: Assoc. Prof. Dr. Tran Minh Triet

Teaching Assistant: Tran Ngoc Dat Thanh

Project team: Huynh Tran Uy - 23125047
Tran My An - 23125049
Nguyen Hoang Gia Han - 23125055
Pham Bao Kha - 23125058

September 2026

Contents

1 Introduction and Game Concept

Zombie House 3D is a third-person, low-poly 3D defense game. The player protects a house from advancing zombie waves by positioning defensive plants, fighting nearby enemies, and managing limited resources. The project combines a readable stylized world with a compact survival loop: prepare a lane, react to enemies, protect the base, and clear all waves.

The game is designed for short sessions and clear feedback. A round is won when all configured waves are defeated; it is lost when the house health reaches zero. Three visual map variants—day, cloudy, and night—support the same core game idea while giving the project a stronger presentation layer.

Design goals. The project is guided by three practical goals:

- Make the defense objective immediately understandable through the house, lanes, enemies, health state, and round UI.
- Combine direct player action with strategic plant placement instead of relying on a purely passive tower-defense experience.
- Use a modular Unity scene and prefab structure so maps, waves, plants, enemies, and UI can be iterated independently.

2 Gameplay and Player Experience

2.1 Core loop

At the start of a round, the player enters a protected area containing planting lanes and a house objective. Sun is collected and spent to deploy plants on valid squares. Zombies enter through configured routes and move toward the house. Plants automatically engage targets in their lane while the player can move, attack, plant, and remove plants. The player adapts the defense until every wave is cleared or the base is destroyed.

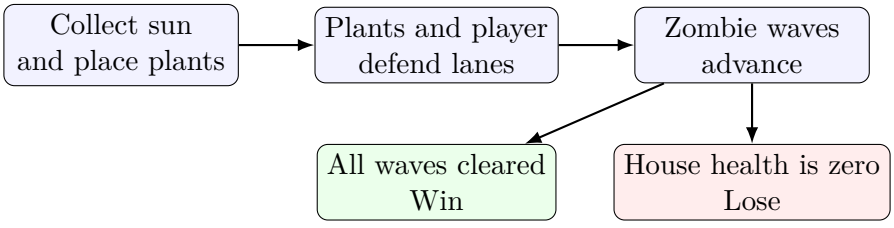

Figure 1: High-level gameplay loop.

2.2 Controls and feedback

The player uses standard third-person movement and camera control. Contextual actions cover selecting a plant, placing it on a free planting square, removing a placed plant, and melee attacking nearby zombies. The UI presents the active round, resource state, base health, and win/lose outcome. Audio reinforces menu navigation, attacks, plant placement, projectile hits, zombie events, resource collection, and round results.

2.3 Tutorial map requirement

The next user-facing addition is a dedicated instruction map. It will teach movement, camera control, plant selection and placement, melee attacks, plant removal, resource collection, and the

win/loss objective in a safe sequence before the player enters a full round.

3 Technical Implementation

3.1 Technology stack

Layer	Technology and role
Game engine	Unity 6.3 LTS with Universal Render Pipeline (URP) for scene management, rendering, prefabs, physics, and UI.
Programming	C# scripts for player input, combat, zombies, waves, health, economy, object pooling, UI state, and audio.
Input and UI	Unity Input System and Unity UI for character selection, round selection, HUD, and win/lose panels.
Version control	Git and GitHub branches for parallel feature development and integration.
Assets	Unity Asset Store packages, KayKit characters, a Cartoon Zombie model, and documented third-party audio.

3.2 Runtime structure

The project separates reusable gameplay systems from map presentation. The main menu selects a round and loads a map scene. Gameplay scenes contain the player, routes, planting area, house target, spawner, grid, and UI systems. Managers coordinate economy, waves, game state, and object pooling, while prefabs keep plants and enemy types reusable.

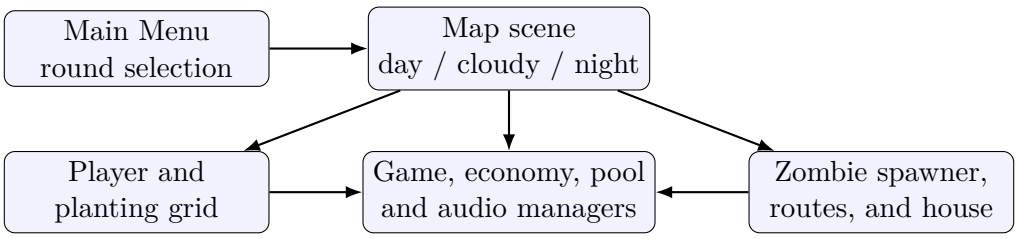

Figure 2: Scene-level organization of the game.

3.3 Audio service

A persistent `AudioManager` selects menu music for `MainMenu` and gameplay music for map scenes. It exposes named one-shot cues for UI and gameplay events and uses several SFX sources so concurrent sounds do not cut each other off. A fallback Audio Listener supports isolated test scenes that do not have a camera.

4 Key Game Features

Figure 3: Project visual used for the main-menu presentation: a protected house facing an approaching zombie threat.

Zombie House 3D is designed around readable, complementary systems rather than a large number of unrelated mechanics. Figure 3 communicates the intended contrast between the player's safe base and the hostile enemy area; the following features turn that setting into a playable defense round.

4.1 Player-driven combat and planting

- **Third-person movement and melee:** the player can move around the map, adjust the camera, and fight enemies that reach the defensive line.
- **Plant-based defense:** plantable squares and plant data allow the player to place defenders deliberately instead of depending only on direct attacks.
- **Resource decisions:** the sun economy connects resource collection and plant costs, requiring the player to decide when and where to invest in defense.
- **Flexible interaction:** selected plants can be placed or removed, which supports correction and adaptation during a round.

4.2 Enemy waves, progression, and feedback

- **Wave defense:** zombies follow routes toward the house, attack when they arrive, and are cleared through coordinated player and plant damage.
- **Multiple enemy forms:** the integration setup includes standard zombie behavior and a spider/zombie variant, making the enemy system extensible.
- **Clear game states:** house health, wave progression, and win/lose panels communicate whether the player is holding the defense or has completed the round.
- **Presentation support:** day, cloudy, and night map variants, a minimap, menu navigation, background music, and action SFX reinforce orientation and feedback.

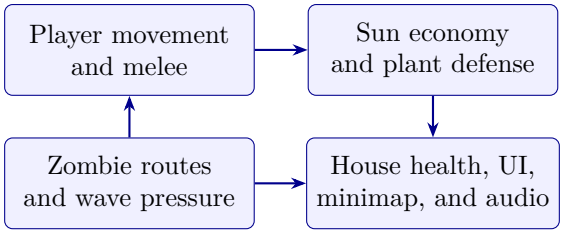

Figure 4: The main gameplay features form one defense loop: enemy pressure drives player and plant actions, while the house and UI provide the round's feedback.

5 Game Features

Feature	Description
Main menu and round flow	Character and round selection route the player to the available map scenes.
Map presentation	Day, cloudy, and night variants provide the project's low-poly outdoor setting.
Player interaction	Third-person movement, camera control, melee attack, plant selection, placement, and removal support direct interaction during a round.
Plants and economy	Plant data, sun economy, planting squares, and Peashooter projectile combat form the defense layer.
Zombie gameplay	Route movement, health, melee attacks, waves, a house target, and spider/zombie enemy variants define the enemy challenge.
Minimap	The minimap follows the player and identifies zombies, helping players remain oriented in the main maps.
Win/lose UI	Result panels and their associated logic communicate the outcome of each round.
Audio	Dedicated menu and gameplay background music, plus UI, combat, plant, zombie, house, resource, and result sound effects support feedback.
Tutorial map	A guided instruction map introduces movement, planting, attacking, plant removal, resource collection, and the defense objective.

Together, these features establish a complete 3D defense-game loop from menu selection to the final win or lose result.

6 Team Contributions and Remaining Work

Member	Student ID	Main contribution
Huynh Tran Uy	23125047	Core gameplay development: player combat, zombie behavior, wave/gameplay integration, and reusable gameplay systems.
Tran My An	23125049	Visual map variants, menu and character-selection UI, and enhancement of the win/lose presentation and round flow.
Nguyen Hoang Gia Han	23125055	Map-route integration support, project audio service and sound library, asset/setup investigation, quality checks, and report preparation.
Pham Bao Kha	23125058	Minimap implementation, project organization, and integration of plantable squares into the map.

Immediate roadmap. The remaining work is organized around a complete, testable game flow:

1. Integrate plantable squares, minimap, house, waves, and enhanced win/lose UI into the principal round maps as one verified gameplay flow.
2. Create a tutorial/instruction map covering movement, planting, attacking, removing plants, resource collection, and objectives.
3. Balance wave counts, plant costs, enemy health, and sound volumes after repeated playtests.
4. Run a final scene-by-scene regression test from Main Menu to win/lose outcomes and verify all third-party asset credits.

7 Conclusion

Zombie House 3D establishes a playable 3D defense-game foundation in Unity: an explorable low-poly setting, round selection, direct player combat, plant-based defense, routed enemies, base health, UI feedback, and an audio layer. The next milestone is not a new game concept but a focused integration pass: place the remaining gameplay systems in the main maps, add the tutorial scene, and tune the complete loop through playtesting. This approach keeps the project achievable while producing a coherent final player experience.

References

[1] Unity Technologies, *Unity Manual and Scripting API*. https://docs.unity3d.com/.

[2] Unity Technologies, *Input System package documentation*. https://docs.unity3d.com/Packages/com.unity.inputsystem@latest.

[3] Unity Technologies, *Universal Render Pipeline documentation*. https://docs.unity3d.com/Manual/com.unity.render-pipelines.universal.html.

[4] Project team, *ASSETS.md: Third-Party Assets*, project repository, accessed September 2026.

[5] KayKit Game Assets, *KayKit Character Pack Adventures*. https://github.com/KayKit-Game-Assets/KayKit-Character-Pack-Adventures-1.0.

[6] Vinrax, *Cartoon Zombie*, CC-BY 3.0; attribution retained in the project asset folder.

[7] TAD, *Once Upon a Time (loop)*, CC0. https://opengameart.org/content/once-upon-a-time-loop.

[8] request, *Heartfelt Battle (loopable fantasy music)*, CC0. https://opengameart.org/content/heartfelt-battle-loopable-fantasy-stringspianohorn.

[9] Kenney, *Interface Sounds* and *RPG Audio*, CC0. https://kenney.nl/assets/interface-sounds; https://kenney.nl/assets/rpg-audio.